



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CAMPUS BIRIGUI
ENGENHARIA DE COMPUTAÇÃO

DOCUMENTAÇÃO DO SISTEMA - FATAL BARCOS

Descrição de sistema computacional

Discentes:

Filipe Camara - BI3041123

Henrique Carvalho - BI3040941

Kauan do Nascimento - BI3043029

Pedro Cabral - BI3042022

Rafael Bueno - BI3042626

Trabalho Prático de Banco de Dados 2 – Submetido ao Professor **Murilo Vargues da Silva** responsável pela disciplina de **Banco de Dados 2**, do curso de **Engenharia de Computação**, **Instituto Federal de Educação, Ciência e Tecnologia de São Paulo – IFSP, Campus Birigui.**

SUMÁRIO

1. DESCRIÇÃO.....	3
2. OBJETIVO GERAL DO SISTEMA.....	3
3. REQUISITOS FUNCIONAIS.....	4
TABELA DE REQUISITOS FUNCIONAIS DO PROJETO.....	4
4. CASOS DE USO PRINCIPAIS.....	9
5. DIAGRAMA FÍSICO DO BANCO DE DADOS.....	10
6. EVIDÊNCIAS DE EXECUÇÃO.....	13

1. DESCRIÇÃO

O sistema foi desenvolvido para a utilização em uma casa de barcos (noturna) que possui barcos para locação. Esse estabelecimento possui seus próprios barcos, mas o negócio principal é a possibilidade de outras empresas detentoras disponibilizarem seus barcos neste estabelecimento. O empreendimento também conta com a possibilidade de aquisição de produtos ou serviços durante as sessões, como cerveja, petiscos, piloto para guiar o passeio ou um marinheiro para auxiliar na pesca.

2. OBJETIVO GERAL DO SISTEMA

O sistema consiste em uma plataforma de gerenciamento para locação e operação de embarcações (Barcos) por sessões de uso. O sistema controla o cadastro de proprietários (Donos), os barcos associados e suas respectivas taxas de comissão. Os Clientes (que obrigatoriamente devem ser maiores de idade) realizam Sessões de uso mediadas por Funcionários cadastrados sob cargos específicos. Durante as sessões, é possível consumir itens ou serviços (Sessão Produto), cujo estoque é deduzido automaticamente via gatilhos do sistema. Ao encerramento da sessão, calcula-se o faturamento com base no tempo decorrido e produtos consumidos.

3. REQUISITOS FUNCIONAIS

TABELA DE REQUISITOS FUNCIONAIS DO PROJETO

CÓDIGO	GRUPO	DESCRIÇÃO	PRIORIDADE	OPERAÇÃO	ATOR
RF-001	Cadastro de Parceiros	Ação: cadastrar (incluir, alterar e excluir) Objeto: Dono Atributos: ID_DONO (INTEGER), CNPJ (18 caracteres), NOME_DONO (50 caracteres) e QNT_BARCOS (INTEGER). Exemplos: {Valtemir de Além car, '00.000.000/0001-00', 0} Restrições/Especificações: - Os atributos CNPJ e NOME_DONO são obrigatórios (NOT NULL). - Não podem ser registrados dois ou mais Donos com o mesmo CNPJ (UNIQUE). - O atributo QNT_BARCOS inicia automaticamente em 0 se não for informado (DEFAULT 0).	Essencial	Entrada	Administrador
RF-002	Cadastro de Atores	Ação: cadastrar (incluir, alterar e excluir) Objeto: Cargo Atributos: ID_CARGO (INTEGER), SALARIO (NUMBER 10,2), CARGA_HORARIA (NUMBER) e NOME_CARGO (50 caracteres). Exemplos: {'Secretário', 2000.00, 44} Restrições/Especificações: - Todos os atributos são estritamente obrigatórios. - O sistema impede o cadastro de dois ou mais cargos com a mesma descrição em NOME_CARGO.	Essencial	Entrada	Administrador

RF-003	Cadastro de Atores	Ação: cadastrar (incluir, alterar e excluir) Objeto: Cliente Atributos: ID_CLIENTE (INTEGER), NOME_CLIENTE (50 caracteres), CPF (14 caracteres), DATA_NASCIMENTO (DATE) e GENERO (2 caracteres). Exemplos: {'Gurila Messias da Silva Bolsonaro', '000.000.001-00', 01/01/1996, 'CM'} Restrições/Especificações: 1. Os atributos NOME_CLIENTE, CPF e DATA_NASCIMENTO são obrigatórios. 2. Não é permitido registrar dois ou mais clientes com o mesmo CPF. 3. Se omitido, o GENERO recebe automaticamente o valor padrão 'N'. 4. O sistema impede o cadastro de clientes menores de 18 anos (TRG_RESTRINGE_IDADE_CLIENTE).	Essencial	Entrada	Funcionário
RF-004	Gestão de Insumos	Ação: cadastrar (incluir, alterar e excluir) Objeto: Produto_Servico Atributos: ID_PRODUTO (INTEGER), NOME_PRODUTO_SERVICO (20 caracteres), PRECO (NUMBER 7,2) e ESTOQUE (NUMBER). Exemplos: {'Cerveja', 7.97, 10} Restrições/Especificações: 1. NOME_PRODUTO_SERVICO e PRECO são obrigatórios. 2. O campo ESTOQUE inicia automaticamente em 0 se não especificado. 3. Não são permitidos produtos diferentes com o mesmo. NOME_PRODUTO_SERVICO.	Essencial	Entrada	Administrador

RF-005	Cadastro de Atores	Ação: cadastrar (incluir, alterar e excluir) Objeto: Funcionário Atributos: ID_FUNCIONARIO (INTEGER), NOME_FUNCIONARIO (50 caracteres), CPF (14 caracteres), ID_CARGO (NUMBER), GENERO (2 caracteres) e DATA_NASCIMENTO (DATE). Exemplos: {'Meu Querido Secretário', '000.000.001-00', 1, 'CM', 01/01/2000} Restrições/Especificações: 1. Todos os campos, exceto gênero, são obrigatórios. O gênero herda o padrão 'N'. 2. O CPF do funcionário deve ser único no sistema. 3. O funcionário deve estar obrigatoriamente associado a um ID_CARGO válido existente. 4. Impedir contratação de menores de 18 anos (TRG_RESTRINGE_IDADE_FUNCIONARIO).	Essencial	Entrada	Adminis trador
RF-006	Gestão de Frota	Ação: cadastrar (incluir, alterar e excluir) Objeto: Barco Atributos: ID_BARCO (INTEGER), NOME_BARCO (50 caracteres), TIE (14 caracteres), ID_DONO (NUMBER), DATA_FABRICACAO (DATE), GENERO (2 caracteres), ORIGEM (20 caracteres), PRECO (NUMBER), COMISSAO (NUMBER) e CAPACIDADE (NUMBER). Exemplos: {'Wandersun', '000.000.001-00', 1, 01/01/1966, 'TW', 'BRASILEIRO', 100.00, 50.00, 3} Restrições/Especificações: 1. Apenas o atributo ORIGEM é opcional, todos os demais são obrigatórios. 2. O número do registro TIE deve ser exclusivo por embarcação. 3. O ID_DONO informado precisa obrigatoriamente existir na tabela Dono. 4. O barco deve ter mais de 18 anos de fabricação (TRG_RESTRINGE_IDADE_BARCOS).	Essencial	Entrada	Adminis trador

RF-007	Operações Náuticas	Ação: iniciar e registrar locação Objeto: Sessão Atributos: ID_SESSAO (INTEGER), ID_BARCO (NUMBER), ID_CLIENTE (NUMBER), ID_FUNCIONÁRIO (NUMBER), VALOR_TOTAL (NUMBER), DESCONTO (NUMBER), INICIO (DATE), FIM (DATE). Exemplos: { 1,1,1,0} Restrições/Especificações: 1. O campo DESCONTO é obrigatório. 2. Os IDs de barco, cliente e funcionário devem referenciar chaves estrangeiras válidas e existentes.	Essencial	Entrada / Proc.	Funcionário
RF-008	Lançamento de Consumo	Ação: vincular itens e atualizar inventário Objeto: Sessão_Produto Atributos: ID_SESSAO_PRODUTO (INTEGER), ID_PRODUTO (NUMBER), ID_SESSAO (NUMBER), VALOR_UNITARIO (NUMBER), QNT (NUMBER). Exemplos: { 1, 1, 7.97, 2} Restrições/Especificações: 1. Os atributos VALOR_UNITARIO e QNT (quantidade) são estritamente obrigatórios. 2. Deve referenciar uma Sessao ativa e um Produto_Servico existente. 3. A inclusão dispara o gatilho TRG_BAIXA_ESTOQUE, que valida se o estoque é suficiente e realiza a baixa automática.	Essencial	Processamento	Funcionário

RF-009	Consultas e Auditorias	Ação: buscar faturamento por período Objeto: Histórico Financeiro Atributos: Dia, Mês, Ano. Exemplos: Informar P_MES = 6 e P_ANO = 2026 para retornar o faturamento total acumulado no período. Restrições/Especificações: 1. O usuário deve preencher os parâmetros temporais exigidos. 2. O sistema processa os registros das sessões finalizadas (FIM IS NOT NULL) utilizando as funções nativas de faturamento (FN_FATURAMENTO_DIA, MES, ANO).	Essencial	Saída	Administrador
RF-010	Operações de Inventário	Ação: incrementar estoque de insumos Objeto: Produto_Servico Atributos: ID_PRODUTO, Quantidade incremental (P_QNT). Exemplos: Chamar a rotina informando o código do produto e somar 15 unidades ao estoque. Restrições/Especificações: 1. O identificador do produto informado deve ser válido. 2. A quantidade fornecida por parâmetro para o incremento deve ser estritamente maior que zero. 3. O processamento é realizado de forma segura por meio da procedure encapsulada AUMENTA_ESTOQUE.	Essencial	Entrada / Proc.	Administrador

4. CASOS DE USO PRINCIPAIS

UC-01: Realizar Locação e Consumo (Fluxo de Sessão)

1. Ator Principal: Funcionário
2. Atores Secundários: Cliente
3. Pré-condições: O Cliente, o Funcionário e o Barco devem estar previamente cadastrados no sistema, e o Barco deve estar disponível.
4. Fluxo Principal (Sucesso):
 - a. O cliente solicita o aluguel de um barco.
 - b. O Funcionário inicia uma nova sessão informando o ID_BARCO, ID_CLIENTE e ID_FUNCIONARIO.
 - c. O sistema valida se o barco está livre e se o cliente é maior de idade.
 - d. O sistema registra a sessão e captura automaticamente a data/hora de início (TRG_INICIA_SESSAO).
 - e. Durante o passeio, o cliente consome itens e ao final da sessão o funcionário os adiciona à conta.
 - f. O sistema valida e dá baixa automática no estoque de produtos a cada item consumido (TRG_BAIXA_ESTOQUE).
 - g. Ao retornar, o funcionário encerra a sessão invocando a procedure de fechamento (Fecha_Conta), que calcula a duração do uso do barco e soma os itens consumidos.
 - h. O sistema exibe o valor total a ser pago e atualiza o status da sessão para finalizada.

UC-02: Cancelamento de Sessão de Locação

1. Ator Principal: Funcionário ou Administrador
2. Pré-condições: Deve existir uma sessão ativa no sistema.
3. Fluxo Principal (Sucesso):
 - a. O usuário solicita o cancelamento de uma sessão de locação informando o P_ID_SESSAO.
 - b. O sistema executa a procedure encapsulada CANCELA_SESSAO.
 - c. O banco de dados faz uma varredura interna (via cursor) em todos os produtos consumidos naquela sessão.
 - d. O sistema realiza o estorno automático, somando as quantidades de volta ao estoque físico de cada mercadoria não consumida (PRODUTO_SERVICO).
 - e. O sistema remove com segurança os registros vinculados e a sessão.
 - f. O sistema confirma o cancelamento bem-sucedido de forma transacional (COMMIT).
4. Fluxo Alternativo (Tratamento de Erro):
 1. Se ocorrer qualquer falha técnica durante o cancelamento, o banco desfaz todas as alterações parciais automaticamente (ROLLBACK), garantindo que o estoque não fique corrompido.

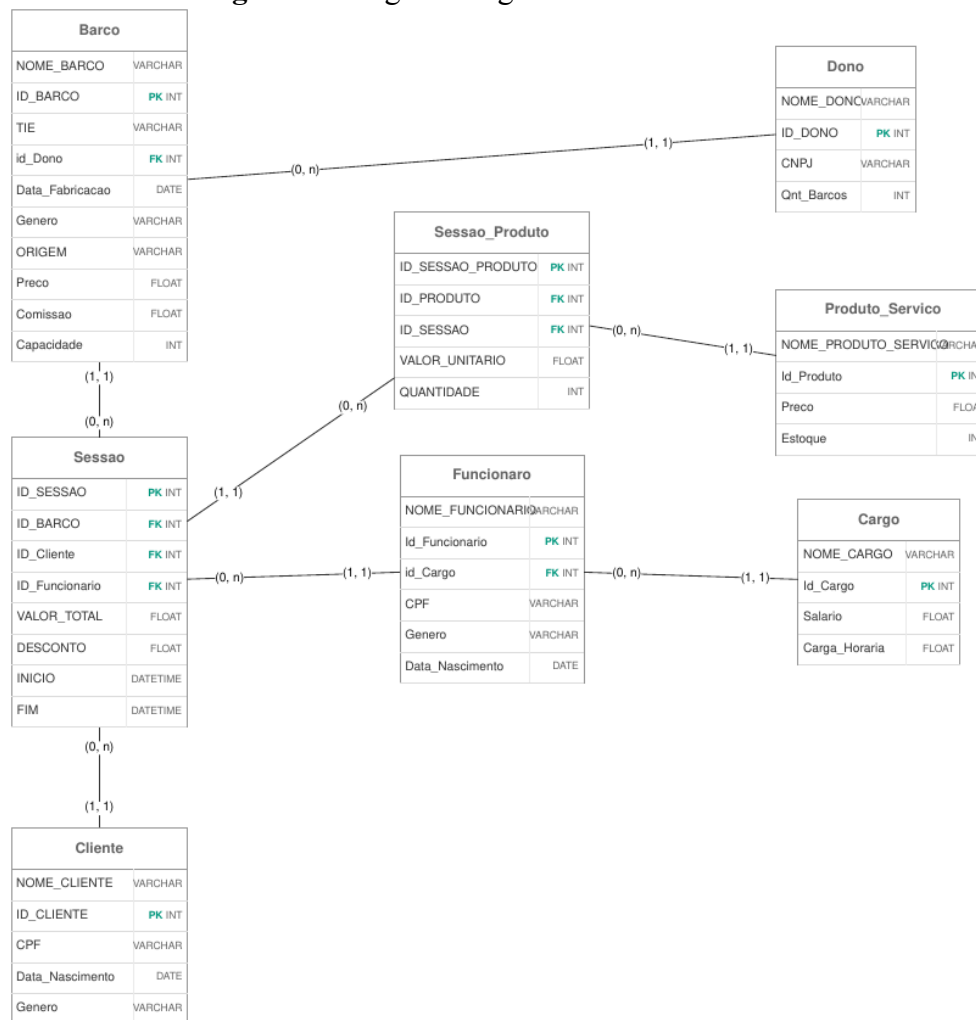
UC-03: Reposição de Inventário e Alerta Crítico

1. Ator Principal: Administrador ou Estoquista
2. Pré-condições: O produto deve estar previamente cadastrado.
3. Fluxo Principal (Sucesso):
 - a. O Administrador identifica a necessidade de reabastecer o estoque e executa a rotina AVISO_REPOSICAO.
 - b. O sistema varre a tabela (C_ESTOQUE) e imprime em tela quais produtos estão com quantidade crítica (≤ 5 unidades).
 - c. O Administrador realiza a compra física dos itens e aciona a procedure AUMENTA_ESTOQUE informando o ID do produto e a quantidade adquirida.
 - d. O sistema valida se a quantidade é maior que zero e se o produto existe.
 - e. O sistema incrementa o estoque e salva a alteração de forma definitiva.

5. DIAGRAMA FÍSICO DO BANCO DE DADOS

Para a elaboração do lógico físico do banco de dados, foi utilizada a ferramenta online BrModelo.

Figura 1: Diagrama lógico do banco de dados



Fonte: Autoria própria

A seguir são apresentados o comando SQL de criação das tabelas (modelo físico do banco de dados):

Tabela Dono:

```
CREATE TABLE Dono
(
    ID_DONO INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    CNPJ VARCHAR2(18) UNIQUE NOT NULL,
    NOME_DONO VARCHAR2(50) NOT NULL,
    QNT_BARCOS INTEGER DEFAULT 0
);
```

Tabela Cargo:

```
CREATE TABLE Cargo
(
    ID_CARGO INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    SALARIO NUMBER(10,2) NOT NULL,
    CARGA_HORARIA NUMBER NOT NULL,
    NOME_CARGO VARCHAR2(50) UNIQUE NOT NULL
);
```

Tabela Cliente:

```
CREATE TABLE Cliente
(
    ID_CLIENTE INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    NOME_CLIENTE VARCHAR2(50) NOT NULL,
    CPF VARCHAR2(14) UNIQUE NOT NULL,
    DATA_NASCIMENTO DATE NOT NULL,
    GENERO VARCHAR2(2) DEFAULT 'N'
);
```

Tabela Produto Serviço:

```
CREATE TABLE Produto_Servico
(
    ID_PRODUTO INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    NOME_PRODUTO_SERVICO VARCHAR2(20) UNIQUE NOT NULL,
    PRECO NUMBER(7,2) NOT NULL,
    ESTOQUE NUMBER DEFAULT 0
);
```

Tabela Barco:

```
CREATE TABLE Barco
(
    ID_BARCO INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    NOME_BARCO VARCHAR2(50) NOT NULL,
    TIE VARCHAR2(14) UNIQUE NOT NULL,
    ID_DONO NUMBER NOT NULL,
    DATA_FABRICACAO DATE NOT NULL,
    GENERO VARCHAR2(2) DEFAULT 'N',
    ORIGEM VARCHAR2(20),
    PRECO NUMBER(6,2) NOT NULL,
    COMISSAO NUMBER(5,2) NOT NULL,
    CAPACIDADE NUMBER NOT NULL,
    FOREIGN KEY (ID_DONO) REFERENCES Dono(ID_DONO)
);
```

Tabela Sessao:

```

CREATE TABLE Sessao
(
    ID_SESSAO INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ID_BARCO NUMBER,
    ID_CLIENTE NUMBER,
    ID_FUNCIONARIO NUMBER,
    VALOR_TOTAL NUMBER(8,2) DEFAULT 0,
    DESCONTO NUMBER NOT NULL,
    INICIO DATE DEFAULT SYSDATE,
    FIM DATE,
    FOREIGN KEY (ID_BARCO) REFERENCES Barco(ID_BARCO),
    FOREIGN KEY (ID_CLIENTE) REFERENCES Cliente(ID_CLIENTE),
    FOREIGN KEY (ID_FUNCIONARIO) REFERENCES Funcionario(ID_FUNCIONARIO)
);

```

Tabela Sessao_Produto:

```

CREATE TABLE Sessao_Produto
(
    ID_SESSAO_PRODUTO INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    ID_PRODUTO NUMBER,
    ID_SESSAO NUMBER,
    VALOR_UNITARIO NUMBER NOT NULL,
    QNT NUMBER NOT NULL,
    FOREIGN KEY (ID_PRODUTO) REFERENCES Produto_Servico(ID_PRODUTO),
    FOREIGN KEY (ID_SESSAO) REFERENCES Sessao(ID_SESSAO)
);

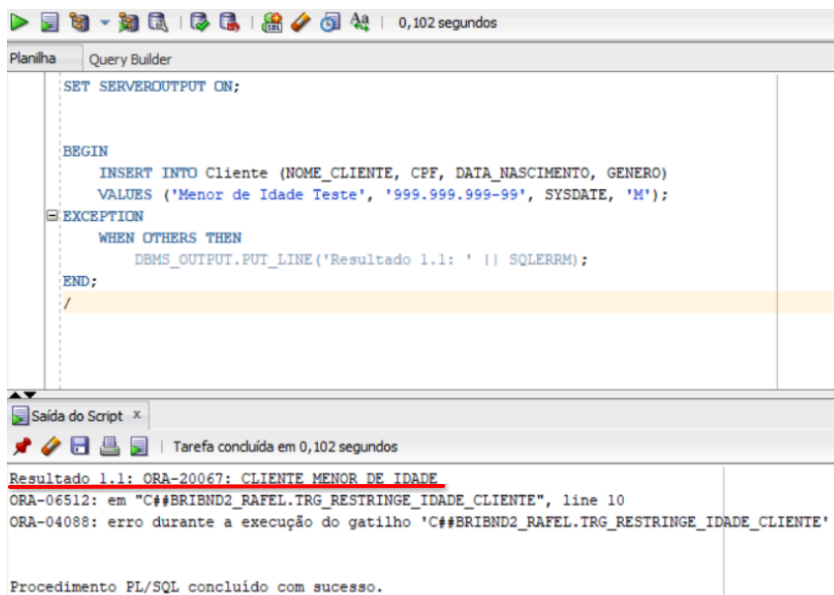
```

6. EVIDÊNCIAS DE EXECUÇÃO

CATEGORIA 1: TESTE DAS TRIGGERS DE MAIORIDADE (RESTRIÇÃO DE IDADE)

Teste 1.1: Tentativa de inserir Cliente Menor de Idade (RF-003)

Esperado: O banco deve bloquear a inserção e disparar o erro: "ORA-20067: CLIENTE MENOR DE IDADE"



```
SET SERVEROUTPUT ON;

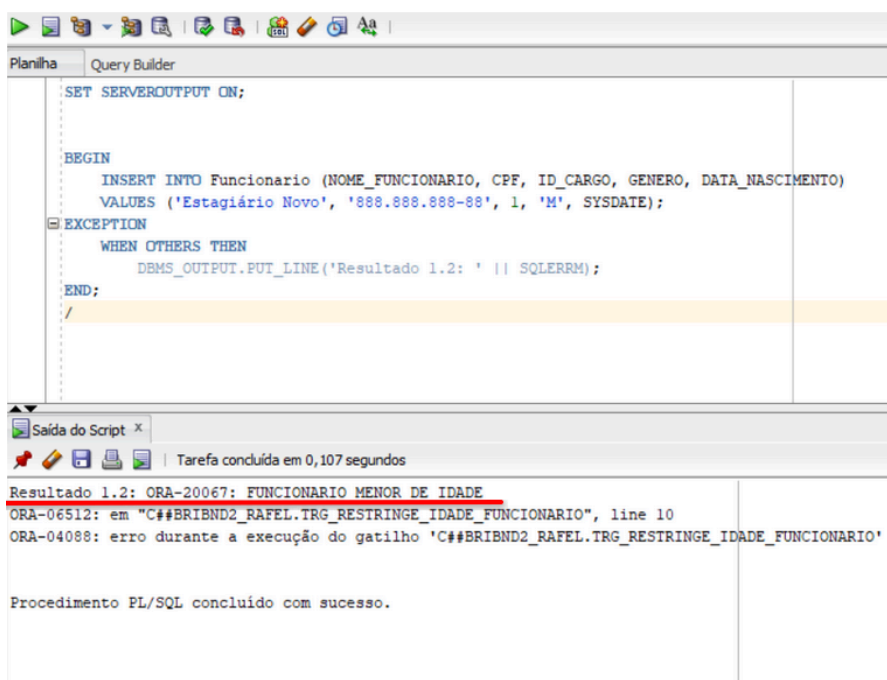
BEGIN
  INSERT INTO Cliente (NOME_CLIENTE, CPF, DATA_NASCIMENTO, GENERO)
  VALUES ('Menor de Idade Teste', '999.999.999-99', SYSDATE, 'M');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Resultado 1.1: ' || SQLERRM);
END;
/
```

Resultado 1.1: ORA-20067: CLIENTE MENOR DE IDADE
ORA-06512: em "C##BRIEND2_RAFEL.TRG_RESTRINGE_IDADE_CLIENTE", line 10
ORA-04088: erro durante a execução do gatilho 'C##BRIEND2_RAFEL.TRG_RESTRINGE_IDADE_CLIENTE'

Procedimento PL/SQL concluído com sucesso.

Teste 1.2: Tentativa de inserir Funcionário Menor de Idade (RF-005)

Esperado: O banco deve bloquear a inserção e disparar o erro: "ORA-20067: FUNCIONÁRIO MENOR DE IDADE"



```
SET SERVEROUTPUT ON;

BEGIN
  INSERT INTO Funcionario (NOME_FUNCIONARIO, CPF, ID_CARGO, GENERO, DATA_NASCIMENTO)
  VALUES ('Estagiário Novo', '888.888.888-88', 1, 'M', SYSDATE);
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Resultado 1.2: ' || SQLERRM);
END;
/
```

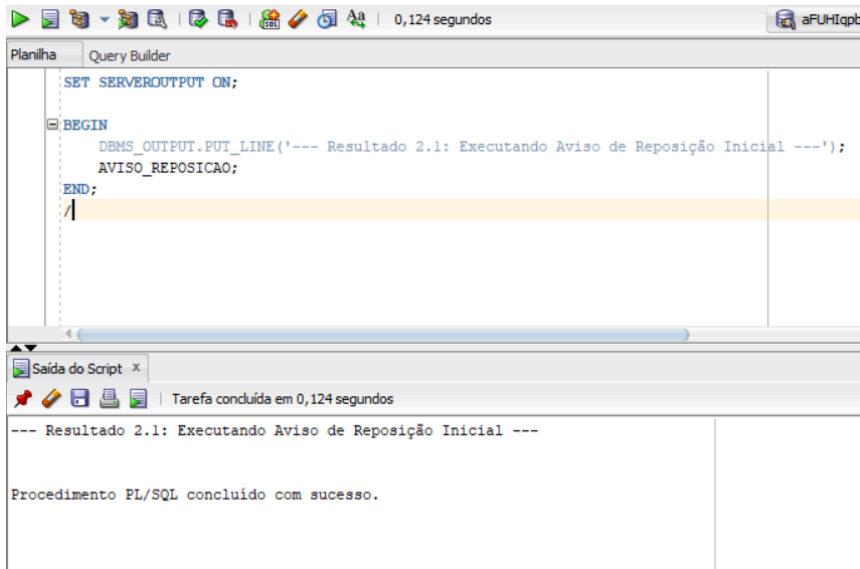
Resultado 1.2: ORA-20067: FUNCIONARIO MENOR DE IDADE
ORA-06512: em "C##BRIEND2_RAFEL.TRG_RESTRINGE_IDADE_FUNCIONARIO", line 10
ORA-04088: erro durante a execução do gatilho 'C##BRIEND2_RAFEL.TRG_RESTRINGE_IDADE_FUNCIONARIO'

Procedimento PL/SQL concluído com sucesso.

CATEGORIA 2: TESTE DE CONTROLE DE ESTOQUE E INVENTÁRIO

Teste 2.1: Executar rotina de verificação de alertas de estoque crítico (UC-003)

Esperado: Como a Cerveja (ID 1) inicia com estoque 10, nenhum alerta deve aparecer ainda.



The screenshot shows the SQL Developer interface. The top toolbar indicates a duration of 0,124 segundos. The 'Planilha' (Worksheet) tab is active, displaying a PL/SQL script in the 'Query Builder' area. The script is as follows:

```
SET SERVEROUTPUT ON;

BEGIN
  DBMS_OUTPUT.PUT_LINE('--- Resultado 2.1: Executando Aviso de Reposição Inicial ---');
  AVISO_REPOSICAO;
END;
```

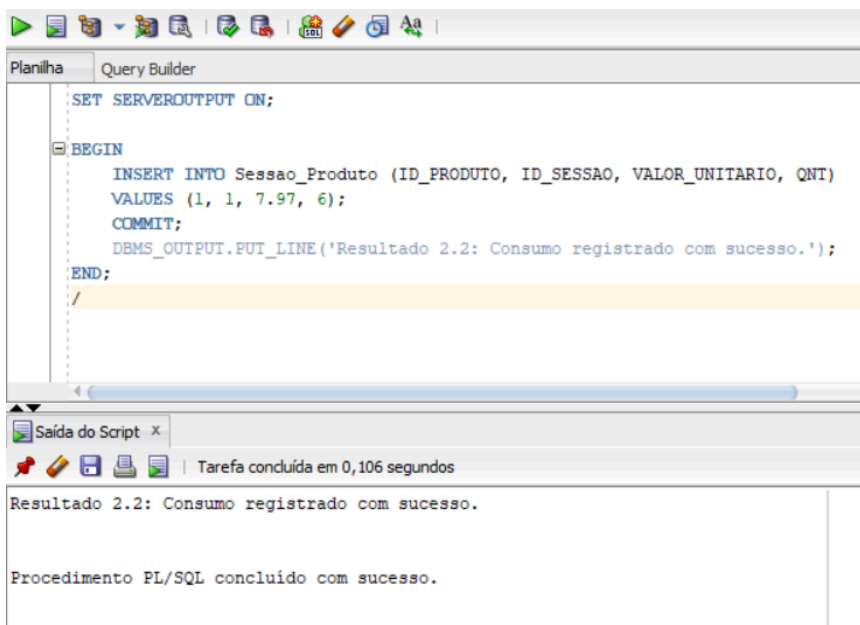
The script is highlighted in yellow. Below the script, the 'Saída do Script' (Script Output) window shows the execution results:

```
--- Resultado 2.1: Executando Aviso de Reposição Inicial ---

Procedimento PL/SQL concluído com sucesso.
```

Teste 2.2: Simular consumo e validar a Trigger de Baixa Automática (RF-008) (TRG_BAIXA_ESTOQUE)

Esperado: Inserir a venda de 6 unidades da Cerveja. O estoque deve cair de 10 para 4.



The screenshot shows the SQL Developer interface. The top toolbar indicates a duration of 0,106 segundos. The 'Planilha' (Worksheet) tab is active, displaying a PL/SQL script in the 'Query Builder' area. The script is as follows:

```
SET SERVEROUTPUT ON;

BEGIN
  INSERT INTO Sessao_Produto (ID_PRODUTO, ID_SESSAO, VALOR_UNITARIO, QNT)
  VALUES (1, 1, 7.97, 6);
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Resultado 2.2: Consumo registrado com sucesso.');
```

The script is highlighted in yellow. Below the script, the 'Saída do Script' (Script Output) window shows the execution results:

```
Resultado 2.2: Consumo registrado com sucesso.

Procedimento PL/SQL concluído com sucesso.
```

Teste 2.3: Consulta de verificação do estoque atual pós-venda

Esperado: O campo ESTOQUE deve retornar exatamente o valor 4.

The screenshot shows a SQL query in a script editor:

```
SELECT NOME_PRODUTO_SERVICO, ESTOQUE
FROM Produto_Servico
WHERE ID_PRODUTO = 1;

COMMIT;
```

Below the script, the 'Resultado da Consulta' (Query Result) tab is active, showing the following data:

NOME_PRODUTO_SERVICO	ESTOQUE
1 Cerveja	4

The status bar indicates 'Todas as Linhas Extraídas: 1 em 0,03 segundos'.

Teste 2.4: Forçar alerta de estoque baixo (≤ 5 unidades)

Esperado: Como o estoque atual é 4, a procedure deve imprimir o alerta no console.

The screenshot shows a PL/SQL procedure being executed in a script editor:

```
SET SERVEROUTPUT ON
BEGIN
  DBMS_OUTPUT.PUT_LINE('--- Resultado 2.4: Executando Aviso com Estoque Baixo ---');
  AVISO_REPOSICAO;
END;
```

The 'Resultado da Consulta' (Query Result) tab is active, showing the following output:

```
--- Resultado 2.4: Executando Aviso com Estoque Baixo ---
REPOR PRODUTO: Cerveja ESTOQUE=4

Procedimento PL/SQL concluído com sucesso.
```

The status bar indicates 'Tarefa concluída em 0,243 segundos'.

Teste 2.5: Forçar estouro de estoque insuficiente (UC-001)

Esperado: Tentar comprar 5 unidades tendo apenas 4. Deve disparar o erro: "ORA-20001: ESTOQUE INSUFICIENTE"

The screenshot shows an INSERT statement in a script editor:

```
INSERT INTO Sessao_Produto (ID_PRODUTO, ID_SESSAO, VALOR_UNITARIO, QNT) VALUES (1, 1, 7.97, 5);
```

Below the script, the 'Resultado da Consulta' (Query Result) tab is active, showing the following error message:

```
Relatório de erros -
Erro de SQL: ORA-20001: ESTOQUE INSUFICIENTE
ORA-06512: em "C##BRIAND2_HENRIQUE.TRG_BAIXA_ESTOQUE", line 11
ORA-04098: erro durante a execução do gatilho 'C##BRIAND2_HENRIQUE.TRG_BAIXA_ESTOQUE'
```

The status bar indicates 'Tarefa concluída em 0,665 segundos'.

Teste 2.6: Entrada/Abastecimento de estoque via Procedure manual

Esperado: Adicionar 20 unidades ao estoque atual (4). O estoque final deve ir para 24.

```
SET SERVEROUTPUT ON
BEGIN
  AUMENTA_ESTOQUE(1, 20);
  DBMS_OUTPUT.PUT_LINE('Resultado 2.6: Procedure AUMENTA_ESTOQUE executada.');
```

Procedimento PL/SQL concluído com sucesso.

Teste 2.7: Consulta de validação pós-abastecimento

Esperado: Deve retornar estoque igual a 24.

```
SELECT NOME_PRODUTO_SERVICO, ESTOQUE
FROM Produto_Servico
WHERE ID_PRODUTO = 1;
```

NOME_PRODUTO_SERVICO	ESTOQUE
1 Cerveja	24

CATEGORIA 3: TESTE DE PROCEDURES DE RH (REAJUSTES SALARIAS)

Teste 3.1: Aplicar reajuste percentual linear de 10% em todos os cargos através de cursor
Esperado: O salário do cargo 'Secretário' deve subir de R\$ 2000.00 para R\$ 2200.00.

```
BEGIN
  REAJUSTA_SALARIOS(10);
  DBMS_OUTPUT.PUT_LINE('Resultado 3.1: Salários reajustados via Cursor com sucesso.');
```

Tarefa concluída em 0,117 segundos

Resultado 3.1: Salários reajustados via Cursor com sucesso.

Procedimento PL/SQL concluído com sucesso.

Teste 3.2: Consulta de verificação do novo salário
Esperado: Visualizar o salário com o acréscimo de 10%.

```
SELECT NOME_CARGO, SALARIO FROM Cargo WHERE ID_CARGO = 1;

COMMIT;
```

Todas as Linhas Extraídas: 1 em 0,035 segundos

	NOME_CARGO	SALARIO
1	Secretário	2200

CATEGORIA 4: FECHAMENTO DE CONTA E LÓGICA DE PORCENTAGEM (IN OUT SESSÃO)

Teste 4.1: Simular a atribuição de um desconto em porcentagem (ex: 10%) na sessão para o teste

```
UPDATE Sessao SET DESCONTO = 10 WHERE ID_SESSAO = 1;

COMMIT;
```

Tarefa concluída em 0,063 segundos

1 linha atualizado.

Teste 4.2: Executar a Procedure Fecha Conta com passagem por referência (%ROWTYPE)
 Esperado: O sistema deve calcular dinamicamente a duração em horas do barco, somar com os produtos consumidos, aplicar os 10% de desconto sobre o subtotal, dar o UPDATE físico na tabela e retornar os dados na memória RAM.

```

DECLARE
  v_sessao_teste Sessao%ROWTYPE;
BEGIN
  SELECT * INTO v_sessao_teste FROM Sessao WHERE ID_SESSAO = 1;

  Fecha_Conta(v_sessao_teste);

  DBMS_OUTPUT.PUT_LINE('--- Resultado 4.2: Resumo do Cupom Fiscal (IN OUT) ---');
  DBMS_OUTPUT.PUT_LINE('Sessão ID: ' || v_sessao_teste.ID_SESSAO);
  DBMS_OUTPUT.PUT_LINE('Data de Encerramento definida: ' || TO_CHAR(v_sessao_teste.Fim, 'DD/MM/YYYY HH24:MI:SS'));
  DBMS_OUTPUT.PUT_LINE('Valor Líquido Final com Desconto Aplicado: R$ ' || v_sessao_teste.Valor_Total);
END;
/

```

Salida do Script x | Tarefa concluída em 0,068 segundos

```

--- Resultado 4.2: Resumo do Cupom Fiscal (IN OUT) ---
Sessão ID: 1
Data de Encerramento definida: 19/06/2026 18:25:51
Valor Líquido Final com Desconto Aplicado: R$ 169,09

Procedimento PL/SQL concluído com sucesso.

```

CATEGORIA 5: INTELIGÊNCIA FINANCEIRA (FUNÇÕES DE FATURAMENTO)

Teste 5.1: Consultar o faturamento consolidado do dia, mês e ano atuais
 Esperado: Retornar o valor total líquido gerado pela Sessão 1 que acabou de ser fechada.

```

SELECT
  FN_FATURAMENTO_DIA(EXTRACT(DAY FROM SYSDATE), EXTRACT(MONTH FROM SYSDATE), EXTRACT(YEAR FROM SYSDATE)) AS FATURAMENTO_HOJE,
  FN_FATURAMENTO_MES(EXTRACT(MONTH FROM SYSDATE), EXTRACT(YEAR FROM SYSDATE)) AS FATURAMENTO_MES_ATUAL,
  FN_FATURAMENTO_ANO(EXTRACT(YEAR FROM SYSDATE)) AS FATURAMENTO_ANO_ATUAL
FROM DUAL;

```

Salida do Script x | Resultado da Consulta x | Todas as Linhas Extraídas: 1 em 0,134 segundos

	FATURAMENTO_HOJE	FATURAMENTO_MES_ATUAL	FATURAMENTO_ANO_ATUAL
1	169,09	169,09	169,09

CATEGORIA 6: CONTROLE TRANSACIONAL (CANCELAMENTO DE SESSÃO)

Teste 6.1: Executar cancelamento completo e transacional da Sessão 1

Esperado: A procedure CANCELA_SESSAO deve remover os vínculos, deletar a sessão e, por meio do cursor interno, estornar as 6 unidades de cerveja consumidas de volta ao estoque físico da tabela Produto_Servico de forma segura.

```
BEGIN
  CANCELA_SESSAO(1);
  DBMS_OUTPUT.PUT_LINE('Resultado 6.1: Sessão cancelada e transações validadas (Estorno e Deleção).');
END;
```

Tarefa concluída em 0,096 segundos

Resultado 6.1: Sessão cancelada e transações validadas (Estorno e Deleção).

Procedimento PL/SQL concluído com sucesso.

Teste 6.2: Validação final do banco de dados pós-cancelamento

Esperado: A consulta da sessão não deve retornar nada (deletada) e o estoque da cerveja deve ter subido de 36 para 40.

Antes:

```
SELECT * FROM Produto_Servico;

INSERT INTO Funcionario (ID_FUNCIONARIO, NOME_FUNCIONARIO, CPF, ID_CARGO, DATA_NASCIMENTO)
VALUES (1, 'João Silva', '123456789', 1, '19/06/2026');
--INSERT INTO Funcionario (NOME_FUNCIONARIO, CPF, ID_CARGO, GENERO, DATA_NASCIMENTO)
VALUES ('João Silva', '123456789', 1, 'M', '19/06/2026');
SELECT * FROM Funcionario;
```

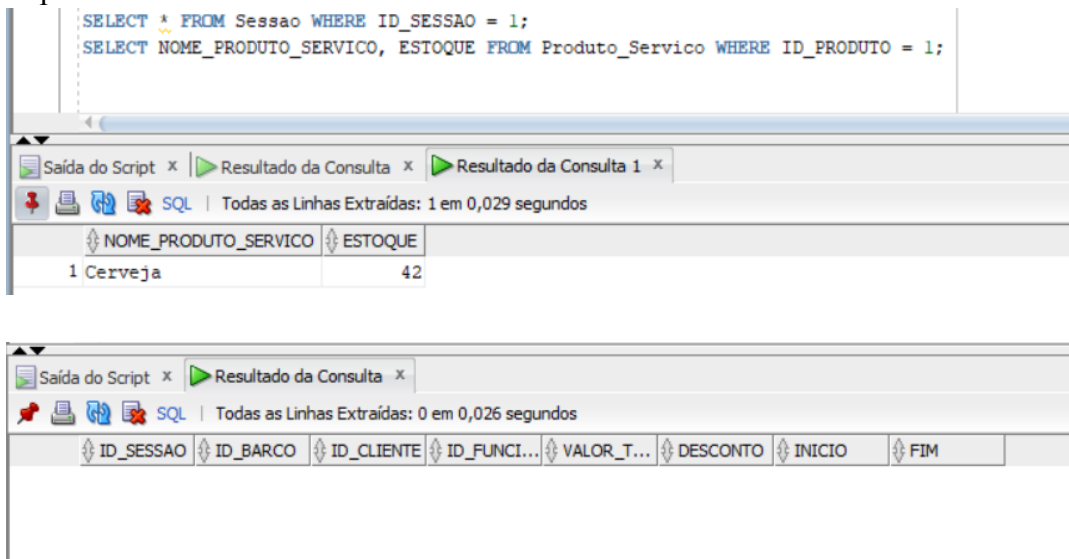
Todas as Linhas Extraídas: 1 em 0,031 segundos

ID_PRODUTO	NOME_PRODUTO_SERVICO	PRECO	ESTOQUE
1	1 Cerveja	7,97	36

Todas as Linhas Extraídas: 1 em 0,026 segundos

ID_SESSAO	ID_BARCO	ID_CLIENTE	ID_FUNCIONARIO	VALOR_TOTAL	DESCONTO	INICIO	FIM
1	1	1	1	1	0	0 19/06/2026	(null)

Depois:



7. CONTRIBUIÇÕES

Todos os membros do grupo auxiliaram na idealização do projeto e na criação do modelo lógico do banco de dados.

- **Filipe Camara:** escreveu o script de criação das tabelas (DDL) e organizou os atributos e relacionamentos das entidades;
- **Henrique Carvalho:** criou as triggers, as functions e os comandos DCL;
- **Kauan do Nascimento:** revisou os códigos e criou inserções para as tabelas;
- **Pedro Cabral:** criou e formatou o documento e organizou as ideias do grupo;
- **Rafael Bueno:** criou as procedures, transações e cursores;
- **Inteligência Artificial:** a inteligência artificial utilizada foi o Gemini do Google. Ela contribuiu na concepção do script de criação das tabelas a partir do diagrama lógico, bem como na otimização de códigos para certas funções e no esclarecimento do funcionamento dos recursos do banco de dados (como os comandos DCL).